Mechatronics System Integration (MCTA3203)

Week 5: L298P Motor Driver Shield with GPIO (ver. 2.0)

Controlling a DC Motor Using L298P Motor Driver Shield and GPIO

Objectives

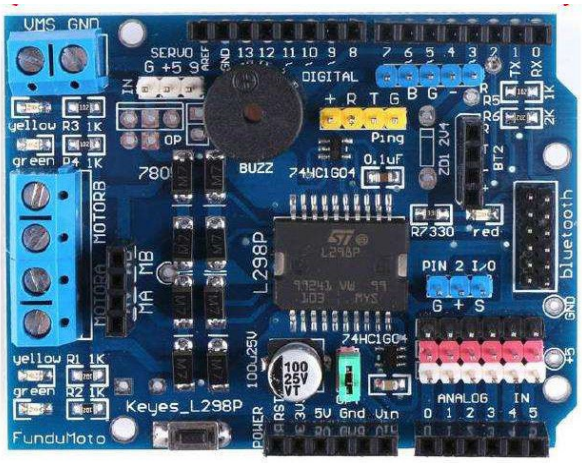
- To understand the working of the L298P Motor Driver Shield.
- To control the speed and direction of a DC motor using GPIO pins.
- To implement PWM (Pulse Width Modulation) for speed control.
- To write and upload code using Arduino IDE

Required Hardware

- Arduino UNO
- L298P Motor Driver Shield
- DC Motor (6V–12V) 1–2
- External Power Supply (9V/12V)
- Jumper Wires
- Breadboard

L298P Motor Driver Shield

- Based on L298P H-Bridge chip.
- Can drive 2 DC motors with direction and speed control.
- Uses digital GPIO pins for direction.
- Uses PWM-capable GPIO pins for speed.



Important Pins on L298P Shield

Function	Arduino Pin Mapping
Motor A Direction	D12 (IN1), D13 (IN2)
Motor B Direction	D10 (IN3), D11 (IN4)
Speed Control A	D3 (ENA - PWM)
Speed Control B	D5 (ENB - PWM)

Note: Actual pin mapping may vary based on shield model (confirm from the datasheet).

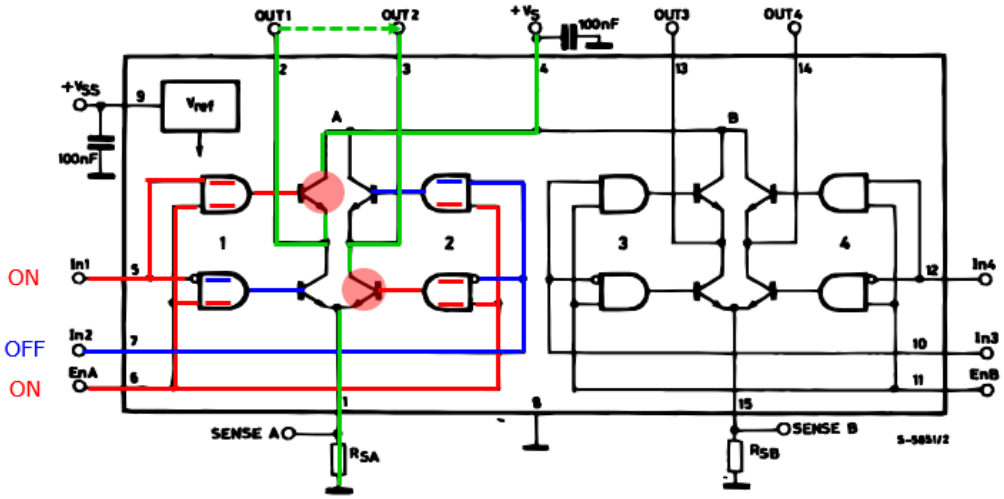
DC Motor

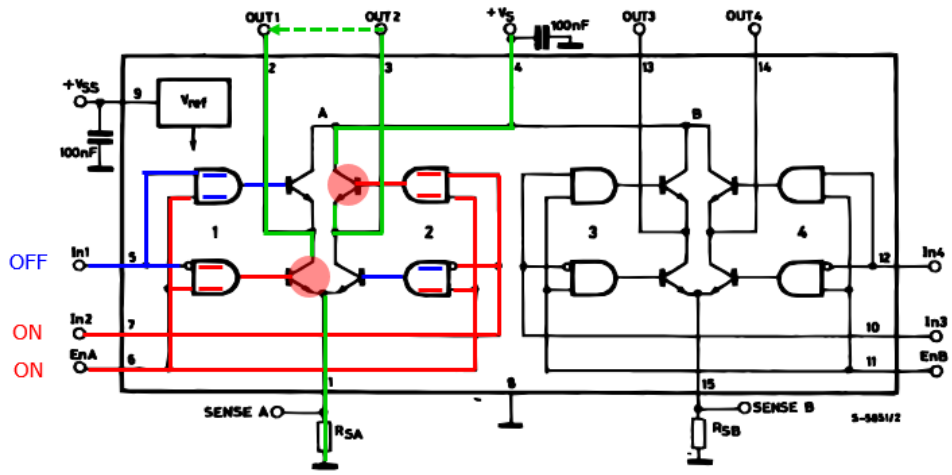
DC Motor Electromechanical Principles DC motors operate based on the Lorentz force principle. When a current-carrying conductor (the armature windings) is placed in a magnetic field (produced by the stator magnets), it experiences a mechanical force that generates torque. The rotational speed of a standard brushed DC motor is directly proportional to the applied voltage, while the torque is proportional to the current drawn. As the motor spins, it acts as a generator, producing a counter-electromotive force (Back EMF) that opposes the supply voltage. The faster the motor spins, the higher the Back EMF, which naturally limits the maximum speed and reduces the current draw as the motor reaches a steady-state velocity.

H-Bridge Topology and Current Routing

The L298P shield is based on an H-Bridge integrated circuit. An H-Bridge is a solid-state circuit utilizing four switching elements (typically transistors) arranged in an "H" shape around the motor load.

- Forward/Reverse Operation: By closing two diagonally opposed switches, current flows through the motor in one direction. Closing the opposite diagonal pair reverses the current flow, reversing the motor's rotational direction. The L298P utilizes digital GPIO pins to dictate this direction.
- Braking vs. Coasting: If both upper or both lower switches are closed simultaneously (e.g., setting both directional pins HIGH or both LOW), the motor terminals are shorted together. This creates dynamic braking, rapidly stopping the motor by dissipating its Back EMF as heat. If all switches are open, the motor simply coasts to a stop.



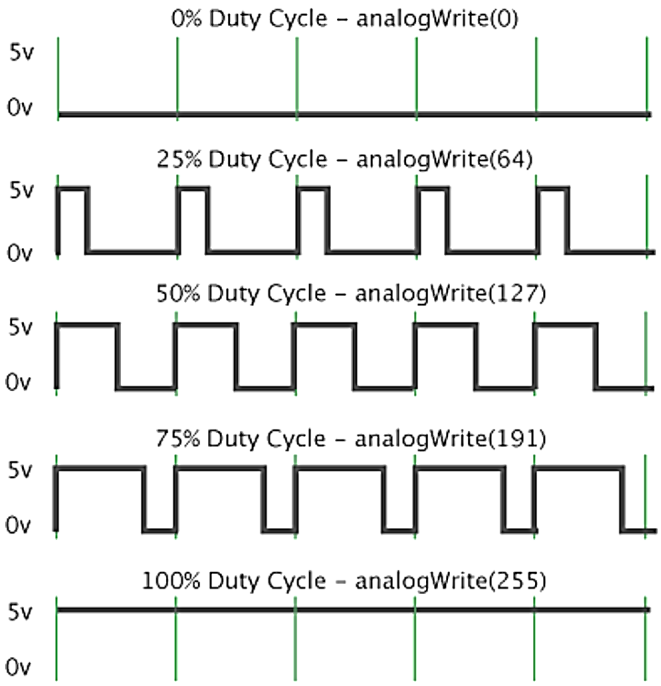


PWM

PWM (Pulse Width Modulation) is a technique that switches the motor's power ON and OFF rapidly at a high frequency. By adjusting the duty cycle — the percentage of time the signal is ON — we control how much average voltage is delivered to the motor. DC motors have high internal inductance that smoothed out the square wave of PWM voltage into a relatively continuous waveform. The frequency of the PWM signal must be fast enough to prevent the motor from juddering during the OFF states.

Arduino PWM can be controlled using *analogWrite(0 to 255)* function to determine the motor's speed:

- Higher duty cycle → higher average voltage → faster rotation
- Lower duty cycle → lower average voltage → slower rotation



Circuit Construction

- Attach the L298P Shield on top of the Arduino board.
- Connect the DC motors to Motor A and Motor B terminals.
- Supply external power to the shield's screw terminal (GND and +12V).
- Ensure GND is common between shield and Arduino.
- For details, please refer to the **references**.

Arduino Code Example

Control Direction and Speed of One Motor

```
// Motor A pin definitions
const int ENA = 3;            // PWM pin for speed control
const int IN1 = 12;           // Direction
const int IN2 = 13;           // Direction

void setup() {
  pinMode(ENA, OUTPUT);
  pinMode(IN1, OUTPUT);
  pinMode(IN2, OUTPUT);
}


void loop() {
  // Move forward
  digitalWrite(IN1, HIGH);
  digitalWrite(IN2, LOW);
  analogWrite(ENA, 100);      // Speed: 0-255

  delay(2000);

  // Move backward
  digitalWrite(IN1, LOW);
  digitalWrite(IN2, HIGH);
  analogWrite(ENA, 100);

  delay(2000);

  // Stop
  analogWrite(ENA, 0);
  delay(1000);
}

void moveMotor(int EN, int IN1, int IN2, int speed, bool forward) {
  // ... will be useful for Task 5
}
```

⚠ Safety Precautions

- Never power motors directly from Arduino 5V pin.
- Always double-check wiring before powering the circuit.
- Disconnect power before making changes.

Experiment Steps

1. Assemble the circuit and plug the L298P shield into the Arduino. Connect a digital multimeter (in DC Voltage mode) in parallel across the Motor A terminals.
2. Upload the base Arduino code and observe the motor rotating in forward and reverse directions.
3. Modify the Arduino code to slowly increase the PWM value from 0 in increments of 5. Record the exact PWM value where the motor overcomes static friction and begins to spin. This establishes the motor's deadband.
4. Modify the PWM value to observe speed changes. For each PWM value in the observation table, record the actual voltage delivered to the motor terminals using the multimeter. Compare this to your external power supply voltage.
5. Write a test sequence that runs the motor at full speed (PWM 255) for 3 seconds, then sets both IN1 and IN2 to LOW (Coasting). Repeat the test, but set both IN1 and IN2 to HIGH (Dynamic Braking). Observe and document the difference in stopping time.

Observation Table

PWM Value	Direction	Observed Speed (RPM est.)*	Motor Terminal Voltage
255	Forward	*??*	*??*
150	Forward	*??*	*??*
75	Forward	*??*	*??*
255	Reverse	*??*	*??*
150	Reverse	*??*	*??*
75	Reverse	*??*	*??*

*Use a tachometer or estimate based on number of rotations per time.

Extended Task

1. Explain the function of the ENA and ENB pins.
2. Describe the reason PWM is used for speed control.
3. Describe the outcome when both IN1 and IN2 are set HIGH.
4. Explain how braking can be implemented using the L298P.
5. Modify the code to control two DC motors simultaneously. Write a custom function driveRobot(int speedLeft, int speedRight) that allows for differential steering.

References

[1] https://www.instructables.com/Tutorial-for-L298-2Amp-Motor-Driver-Shield-for-Ard/
Tutorial for L298 2Amp Motor Driver Shield for Arduino

[2] https://github.com/CytronTechnologies/L298P_MotorDriverShield
L298P_MotorDriverShield

[3] https://my.cytron.io/p-shield-l298p-motor-driver-with-gpio?r=1
Shield L298P Motor Driver with GPIO

[4] https://docs.cirkitdesigner.com/component/a25f6e70-294e-42fd-b77c-9e9349b76b9a/l298p-drive-shield
How to Use L298P drive shield: Examples, Pinouts, and Specs

Appendix

Pin Reference (Arduino UNO)

Shield Pin	Arduino Pin	Function
ENA	D3	PWM (Speed A)
IN1	D12	Motor A Direction 1
IN2	D13	Motor A Direction 2
ENB	D5	PWM (Speed B)
IN3	D10	Motor B Direction 1
IN4	D11	Motor B Direction 2